



POLITECHNIKA MORSKA – WYDZIAŁ INFORMATYKI

INFORMATYKA – LAB 13

Student: Paweł Dutkiewicz

Grupa: L01

1. Opis reprezentacji danych:

Dane w programie prezentowane są za pomocą listy struktur. Każdy element list jest przedstawiony w postaci **student(Imie,Punkty)** gdzie:

- **Imie** - jest daną identyfikacyjną danego studenta,
- **Punkty** - wartość liczbowa **0-20** reprezentująca wynik, lub pozwalająca określić czy dany student zaliczył dane laboratorium.
 - np. **student(anna,18)**, cała baza danych jest przechowywana w predykanie **student(Lista)**, co pozwala na pobranie wszystkich potrzebnych i wykorzystywanych danych przy pomocy jednego zapytania i przekazanie ich dalej do predykatów analizujących przy pomocy zmiennej.

2. Testy programu:

a. Test_1: Suma i liczba studentów

- Zapytanie:
 - **studenci(L), suma_punktow(L, S), liczba_studentow(L, N).**
- Otrzymany wynik:

```
2 ?- studenci(L), suma_punktow(L, S), liczba_studentow(L, N).  
L = [student(anna, 18), student(jan, 9), student(piotr, 15), student(ewa, 20), student(karol, 7), student(maria, 13), student(tomasz, 10), student(alicja, 20)],  
S = 112,  
N = 8.
```

- Komentarz:
 - Program zsumował punkty wszystkich 8 osób z bazy danych zgodnie z poleceniem

b. Test_2: Średnia Punktów

- Zapytanie:
 - **studenci(L),srednia_punktow(L,Srednia).**
- Otrzymany wynik:

```
3 ?- studenci(L),srednia_punktow(L,Srednia).  
L = [student(anna, 18), student(jan, 9), student(piotr, 15), student(ewa, 20), student(karol, 7), student(maria, 13), student(tomasz, 10), student(alicja, 20)],  
Srednia = 14.
```

- Komentarz: Średnia została obliczona jako $112 / 8$, dlatego 14 jest liczbą zgodna z wynikiem

c. Test_3: Liczba studentów, którzy zaliczyli zadanie i tych, którzy go nie zaliczyli

- Zapytanie:
 - **studenci(L),zaliczeni(L,Z),niezaliczeni(L,NZ).**
- Otrzymany wynik:

```
7 ?- studenci(L),zaliczeni(L,Z),niezaliczeni(L,NZ).  
L = [student(anna, 18), student(jan, 9), student(piotr, 15), student(ewa, 20), student(karol, 7), student(maria, 13), student(tomasz, 10), student(alicja, 20)],  
Z = [student(anna, 18), student(piotr, 15), student(ewa, 20), student(maria, 13), student(tomasz, 10), student(alicja, 20)],  
NZ = [student(jan, 9), student(karol, 7)]
```

- Komentarz: Jan i Karol jako jedyni uzyskali wynik poniżej 10 punktów, przez co trafili na liste osob, ktorzy nie zaliczyli zadania, Tomasz ktory był Edge Case'm (miał 10 punktów), trafił do listy osób, którzy zaliczyli zadanie

d. Test_4: Najlepsi studenci:

- Zapytanie:
 - **studenci(L), najlepszy_wynik(L,M), najlepsi_studenci(L,Naj).**
- Otrzymany wynik:

```
8 ?- studenci(L), najlepszy_wynik(L,M),najlepsi_studenci(L,Naj).  
L = [student(anna, 18), student(jan, 9), student(piotr, 15), student(ewa, 20), student(karol, 7), student(maria, 13), student(tomasz, 10), student(alicja, 20)],  
M = 20,  
Naj = [student(ewa, 20), student(alicja, 20)]
```

- Komentarz: Program poprawnie wyznaczył wynik oraz osoby, które go uzyskały
- e. Czy wynik jest zgodny z oczekiwaniem:
Tak wyniki programu, są zgodne z oczekiwaniami.

3. Wnioski:

- a. czym Prolog się różni w przetwarzaniu list w językach imperatywnych:
Przetwarzanie list w prologu znacząco różni się od innych języków imperatywnych jak C++, Python czy Java w poniższych aspektach:
- Sposób dostępu : Head/Tail - w językach imperatywnych stosujemy dostęp przez indeksy np. lista[i] lub lista[0]. W prologu jedyną opcją dostępu do danych jest poprzez oddzielenie pierwszego elementu (Head) od reszty listy (Tail) przy pomocy zapisu [H|T]
 - Rekurencja zamiast pętli - w C++/Pythonie mamy różnego rodzaju pętle jak while, do while, for each etc. Prolog nie posiada natomiast tego typu struktur, każda operacja na liście wymaga rekurencyjnego wywołania predykatu dla Tail'a listy, aż do napotkania listy pustej, która jest warunkiem, który przerywa działanie programu.
 - Niezmienność danych - w językach jak Java można modyfikować elementy listy. W Prologu nie ma takiej możliwości, istnieje opcja jedynie zbudowania nowej listy element po elemencie, na bazie wyniku danej operacji np. filtrowania elementów w liście, ale oryginalna lista nie będzie w żaden sposób zmodyfikowana
 - Dopasowanie wzorców - Prolog pozwala na wyciąganie konkretnych danych ze struktur wewnątrz listy na etapie definiowania argumentów np. [student(_,P)|T]), co w językach imperatywnych wymagałoby dodatkowej instrukcji